INTERNSHIP PROJECT REPORT

STUDENT COURSE MANAGEMENT SYSTEM

Java Spring Boot + Thymeleaf + HTML

Internship Project 1

Academic Year: 2026

CERTIFICATE

This is to certify that the internship project report entitled “Student Course Management System” has been prepared as part of the internship project work. The project demonstrates the development of a web-based student and course management application using Java Spring Boot for the backend, Thymeleaf for server-side page rendering, and HTML/CSS for the user interface.

The project includes functionality for creating and managing student records, associating students with courses, viewing student information, and displaying assigned courses. The report documents the requirements, design, implementation methodology, source code, testing process, and sample results.

ACKNOWLEDGEMENT

I sincerely express my gratitude to my internship mentors, teachers, and everyone who supported me during the development of the Student Course Management System. Their guidance helped me understand how backend services, database persistence, server-side templates, and HTML interfaces can be combined into a complete web application.

This project gave me practical experience with Spring Boot, Spring MVC, Spring Data JPA, Thymeleaf, HTML, CSS, and relational database concepts. I am also thankful for the Java and Spring ecosystem, which provides reusable frameworks and tools for developing maintainable applications.

Finally, I appreciate the opportunity to work on a project that connects programming concepts with a real-world management problem. Developing the system improved my understanding of CRUD operations, MVC architecture, form handling, data persistence, and web application testing.

TABLE OF CONTENTS

1. ABSTRACT

The Student Course Management System is a web-based application developed using Java Spring Boot, Thymeleaf, HTML, CSS, and a relational database. The system is designed to simplify the management of student information and their associated course enrollment. It provides a structured interface through which an administrator or authorized user can create student records, manage their details, assign courses, and view the courses associated with each student.

The backend follows the Spring MVC architecture. Spring Boot is used to configure and run the application, controllers handle browser requests, service classes contain business logic, and repositories communicate with the database using Spring Data JPA. Thymeleaf is used to render dynamic HTML pages on the server side.

The project demonstrates the complete flow of information from a web form to the Java backend and finally into persistent storage. It also demonstrates CRUD-style operations and the relationship between students and courses. The application is suitable as an internship-level project because it combines Java programming, web development, database concepts, and software architecture.

The final system can be expanded with authentication, role-based access, course creation screens, search and filtering, pagination, REST APIs, validation, reporting, and deployment to a cloud platform.

2. OBJECTIVE

The main objective of the project is to build a Student Course Management System using Java Spring Boot with Thymeleaf and HTML for the user interface.

- Create and manage student records through a web interface.
- Store student information persistently in a relational database.
- Associate students with course information.
- Allow users to view courses assigned to a particular student.
- Implement backend logic using Spring Boot and Spring MVC.
- Use Spring Data JPA for database access and entity persistence.
- Use Thymeleaf to generate dynamic server-side HTML pages.
- Create simple and understandable forms for student data entry.
- Validate and test the major application workflows.
- Build a foundation that can be extended into a complete academic management platform.

3. INTRODUCTION

Educational institutions handle large amounts of student and course information. When these records are maintained manually, it becomes difficult to keep information consistent, find records quickly, and understand which courses have been assigned to each student. A web-based management system can centralize this information and make routine operations more efficient.

The Student Course Management System addresses this problem by providing a browser-based interface connected to a Java backend. Spring Boot reduces configuration overhead and provides a structured framework for creating the application. Spring MVC separates web requests from business logic, while Spring Data JPA provides an abstraction over database operations.

Thymeleaf is used because it integrates naturally with Spring MVC and allows Java model data to be inserted into HTML templates. The resulting pages are rendered on the server and returned to the browser. HTML forms provide the user interaction layer, while CSS can be used to improve presentation.

The project therefore provides a practical example of how a Java-based full-stack web application can be developed using a modern but beginner-friendly technology stack.

4. PROBLEM STATEMENT AND SCOPE

The problem is to create a centralized application that can maintain student details and their associated course information. The system should reduce repetitive manual work and provide an easy method for viewing student-course relationships.

Functional Scope

- Create a new student with name, email, and selected course information.
- Display a list of registered students.
- View the courses assigned to a student.
- Update or delete student records in an extended implementation.
- Persist information in a relational database.
- Provide user-friendly success and error messages.

Non-Functional Scope

- Simple and readable user interface.
- Maintainable Java package structure.
- Reasonable separation of controller, service, repository, and entity layers.
- Database persistence using JPA.
- Application should run locally using a Java-supported environment.

5. TECHNOLOGY REQUIREMENTS

Hardware Requirements

- Desktop or laptop computer.
- At least 4 GB RAM recommended for comfortable Spring Boot development.
- At least 500 MB free storage for JDK, IDE, Maven dependencies, and project files.
- Keyboard, mouse/touchpad, and a modern web browser.

Software Requirements

- Java JDK 17 or later.
- Spring Boot 3.x or compatible version.
- Maven for dependency management and project execution.
- IntelliJ IDEA, Eclipse, Spring Tool Suite, or VS Code.
- MySQL, PostgreSQL, or H2 database.
- Google Chrome, Microsoft Edge, Firefox, or another modern browser.

6. TECHNOLOGY OVERVIEW

Java

Java is the core programming language used for the application. Its object-oriented model supports classes, interfaces, dependency injection patterns, exception handling, and reusable business logic.

Spring Boot

Spring Boot simplifies the creation of production-style Java applications by providing auto-configuration, starter dependencies, embedded server support, and a standard application structure.

Spring MVC

Spring MVC handles HTTP requests through controllers. A controller can receive form data, call a service method, add objects to the model, and return a Thymeleaf view.

Spring Data JPA

Spring Data JPA provides repository interfaces for persistence operations. Instead of writing repetitive SQL for basic CRUD operations, the developer can use methods such as save(), findAll(), findById(), and deleteById().

Thymeleaf

Thymeleaf is a server-side Java template engine. It allows dynamic values and iteration to be inserted into HTML templates using attributes such as th:text, th:each, th:object, and th:field.

HTML/CSS

HTML provides the structure of the web pages, while CSS can provide layout, spacing, typography, forms, tables, and visual feedback.

7. SYSTEM REQUIREMENTS

The system is organized around student management and course association. A typical student record contains a unique identifier, name, email address, and course information. For a normalized implementation, courses can be represented as a separate entity and linked to students using a relationship such as many-to-many.

Core Modules

- Student Module – creates, reads, updates, and deletes student records.
- Course Module – stores course names and course identifiers.
- Enrollment Module – associates students with selected courses.
- View Module – displays student details and assigned courses.
- Database Module – persists entities using JPA.
- Web UI Module – renders forms, lists, and result pages using Thymeleaf.

Expected User Flow

Open application → Student Management → Add Student → Enter details → Select course(s) → Save → Student list → Open student details → View assigned courses.

8. METHODOLOGY

The project follows a layered MVC-oriented methodology. The first stage is requirement analysis, followed by database modeling, backend implementation, UI development, integration, and testing.

Step 1 – Requirement Analysis

The required information and operations are identified: student registration, course association, student listing, and assigned-course viewing.

Step 2 – Data Modeling

Student and Course entities are designed. A student can be associated with one or multiple courses depending on the chosen database relationship.

Step 3 – Backend Development

Entity, repository, service, and controller classes are created. The controller maps HTTP requests to Java methods, while the service layer handles application logic.

Step 4 – UI Development

Thymeleaf templates are created for student forms, lists, and course details. HTML forms submit data to Spring MVC endpoints.

Step 5 – Integration

The browser, controller, service, repository, JPA, and database are connected into one workflow.

Step 6 – Testing

The application is tested for student creation, listing, course display, invalid input, and persistence.

9. SYSTEM ARCHITECTURE

The application uses a layered architecture that separates responsibilities. This improves maintainability and makes it easier to test and modify individual components.

- Presentation Layer: Thymeleaf templates, HTML, and CSS.
- Controller Layer: Spring MVC controllers that map browser requests.
- Service Layer: Business rules and coordination between components.
- Repository Layer: Spring Data JPA repositories for database access.
- Entity Layer: Java classes representing Student and Course records.
- Database Layer: Relational database storing persistent application data.

The browser sends an HTTP request to a controller. The controller calls a service, which may call a repository. The repository communicates with the database through JPA/Hibernate. The returned data is placed into a model and rendered by Thymeleaf into an HTML response.

10. DATABASE DESIGN

The database is designed to store student and course information in structured form. A simple implementation may keep a course field directly inside the Student entity, but a more scalable design separates Course into its own table.

Student Table

Field	Type	Description
id	Long	Primary key
name	VARCHAR	Student name
email	VARCHAR	Student email
course	VARCHAR	Assigned course in simple version

For a normalized version, the Course table can contain id, courseCode, and courseName. A student_course join table can then represent many-to-many enrollment. This makes it possible for one student to have multiple courses and one course to have many students.

11. UI DESIGN AND USER FLOW

The user interface is designed around simple forms and readable lists. The home page can provide navigation to student registration and student listing.

Student Form

- Name input field.
- Email input field.
- Course selection field.
- Save button.
- Reset/cancel control where appropriate.

Student List

The student list displays registered students in a table. Each row can include student ID, name, email, course, and an action link for viewing details.

Assigned Courses

The assigned-course view receives the selected student and displays the courses associated with that student. In a multi-course design, Thymeleaf iterates over the course collection.

12. SPRING BOOT PROJECT STRUCTURE

A recommended project structure is shown below. The exact package names may be adjusted according to the internship environment.

```
student-course-management/
├── src/main/java/com/example/scms/
│   ├── StudentCourseManagementApplication.java
│   ├── controller/
│   │   └── StudentController.java
│   ├── entity/
│   │   ├── Student.java
│   │   └── Course.java
│   ├── repository/
│   │   ├── StudentRepository.java
│   │   └── CourseRepository.java
│   └── service/
│       └── StudentService.java
│
├── src/main/resources/
│   ├── templates/
│   │   ├── index.html
│   │   ├── students.html
│   │   ├── student-form.html
│   │   └── student-courses.html
│   └── application.properties
│
└── pom.xml
```

This organization separates web requests, persistence, business logic, entities, and presentation templates. Such separation is useful for internship projects because it demonstrates an understanding of maintainable application architecture.

13. IMPLEMENTATION DETAILS

The implementation starts with the Spring Boot main class. The application is launched using SpringApplication.run(). Spring Boot automatically creates the application context and embedded web server based on the project dependencies.

The Student entity is annotated with @Entity so that JPA can persist it. The repository extends JpaRepository<Student, Long>, providing standard CRUD operations. The service layer calls the repository and can be expanded with business rules.

The controller exposes routes such as /students, /students/new, /students/save, and /students/{id}/courses. The Thymeleaf templates bind form fields to Java objects and render returned model data.

For a production-ready version, validation annotations such as @NotBlank and @Email can be added. Global exception handling, authentication, authorization, pagination, logging, and CSRF protection should also be considered.

14. CODE – ENTITY AND REPOSITORY

The Student entity represents the persistent student record. The repository interface provides database operations without requiring manual SQL for basic CRUD functionality.

Student.java

```
package com.example.scms.entity;

import jakarta.persistence.*;

@Entity
@Table(name = "students")
public class Student {

    @Id
    @GeneratedValue(strategy = GenerationType.IDENTITY)
    private Long id;

    private String name;
    private String email;
    private String course;

    public Student() {
    }

    public Student(String name, String email, String course) {
        this.name = name;
        this.email = email;
        this.course = course;
    }

    public Long getId() {
        return id;
    }

    public String getName() {
        return name;
    }

    public void setName(String name) {
        this.name = name;
    }

    public String getEmail() {
        return email;
    }

    public void setEmail(String email) {
        this.email = email;
    }

    public String getCourse() {
        return course;
    }
```

```
    public void setCourse(String course) {
        this.course = course;
    }
}
```

StudentRepository.java

```
package com.example.scms.repository;

import com.example.scms.entity.Student;
import org.springframework.data.jpa.repository.JpaRepository;

public interface StudentRepository
        extends JpaRepository<Student, Long> {
}
```

15. CODE – SERVICE AND CONTROLLER

The service layer provides an abstraction over the repository. The controller receives HTTP requests and maps them to the required service operations.

StudentService.java

```
package com.example.scms.service;

import com.example.scms.entity.Student;
import com.example.scms.repository.StudentRepository;
import org.springframework.stereotype.Service;

import java.util.List;

@Service
public class StudentService {

    private final StudentRepository repository;

    public StudentService(StudentRepository repository) {
        this.repository = repository;
    }

    public List<Student> getAllStudents() {
        return repository.findAll();
    }

    public Student getStudent(Long id) {
        return repository.findById(id).orElse(null);
    }

    public Student saveStudent(Student student) {
        return repository.save(student);
    }

    public void deleteStudent(Long id) {
        repository.deleteById(id);
    }
}
```

StudentController.java

```
package com.example.scms.controller;

import com.example.scms.entity.Student;
import com.example.scms.service.StudentService;
import org.springframework.stereotype.Controller;
import org.springframework.ui.Model;
import org.springframework.web.bind.annotation.*;

@Controller
@RequestMapping("/students")
public class StudentController {
```

```
    private final StudentService service;

    public StudentController(StudentService service) {
        this.service = service;
    }

    @GetMapping
    public String listStudents(Model model) {
        model.addAttribute("students",
                service.getAllStudents());
        return "students";
    }

    @GetMapping("/new")
    public String showForm(Model model) {
        model.addAttribute("student", new Student());
        return "student-form";
    }

    @PostMapping("/save")
    public String saveStudent(@ModelAttribute Student student) {
        service.saveStudent(student);
        return "redirect:/students";
    }

    @GetMapping("/{id}/courses")
    public String viewCourses(@PathVariable Long id,
                              Model model) {
        Student student = service.getStudent(id);
        model.addAttribute("student", student);
        return "student-courses";
    }
}
```

16. CODE – THYMELEAF TEMPLATES

Thymeleaf binds HTML form elements to Java model attributes. The student form uses th:object and th:field to connect input controls to the Student object. The list page uses th:each to iterate through the students returned by the controller.

```
<!-- student-form.html -->
<!DOCTYPE html>
<html xmlns:th="http://www.thymeleaf.org">
<head>
    <title>Add Student</title>
</head>
<body>
<h1>Add Student</h1>

<form th:action="@{/students/save}"
      th:object="${student}" method="post">

    <label>Name:</label>
    <input type="text" th:field="*{name}" required>

    <label>Email:</label>
    <input type="email" th:field="*{email}" required>

    <label>Course:</label>
    <input type="text" th:field="*{course}" required>

    <button type="submit">Save Student</button>
</form>
</body>
</html>



<!-- students.html -->
<!DOCTYPE html>
<html xmlns:th="http://www.thymeleaf.org">
<head>
    <title>Students</title>
</head>
<body>
<h1>Student List</h1>

<a th:href="@{/students/new}">Add Student</a>

<table border="1">
    <tr>
        <th>ID</th>
        <th>Name</th>
        <th>Email</th>
        <th>Course</th>
        <th>Action</th>
    </tr>

    <tr th:each="student : ${students}">
        <td th:text="${student.id}"></td>
        <td th:text="${student.name}"></td>
```

```
        <td th:text="${student.email}"></td>
        <td th:text="${student.course}"></td>
        <td>
            <a th:href="@{/students/{id}/courses(id=${student.id})}">
                View Course
            </a>
        </td>
    </tr>
</table>
</body>
</html>
```

17. CODE – APPLICATION CONFIGURATION

For a simple internship demonstration, H2 can be used as an embedded relational database. The following configuration allows the application to run without installing a separate database server. The same JPA code can later be connected to MySQL or PostgreSQL by changing the datasource configuration.

application.properties

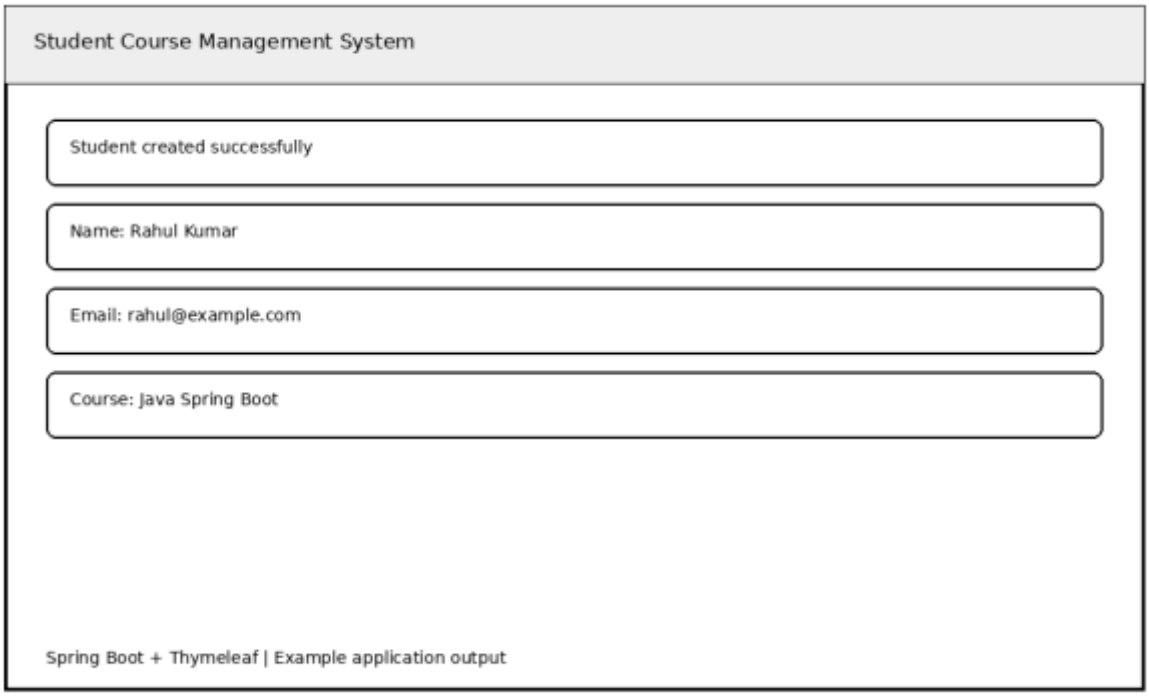
```
spring.application.name=student-course-management

server.port=8080

spring.datasource.url=jdbc:h2:mem:scms
spring.datasource.driverClassName=org.h2.Driver
spring.datasource.username=sa
spring.datasource.password=

spring.jpa.hibernate.ddl-auto=update
spring.jpa.show-sql=true

spring.h2.console.enabled=true
spring.thymeleaf.cache=false
```

Important Maven Dependencies

```
<dependency>
    <groupId>org.springframework.boot</groupId>
    <artifactId>spring-boot-starter-web</artifactId>
</dependency>

<dependency>
    <groupId>org.springframework.boot</groupId>
    <artifactId>spring-boot-starter-thymeleaf</artifactId>
</dependency>

<dependency>
    <groupId>org.springframework.boot</groupId>
    <artifactId>spring-boot-starter-data-jpa</artifactId>
</dependency>

<dependency>
    <groupId>com.h2database</groupId>
    <artifactId>h2</artifactId>
    <scope>runtime</scope>
</dependency>
```

18. RESULTS AND OUTPUT – STUDENT CREATION

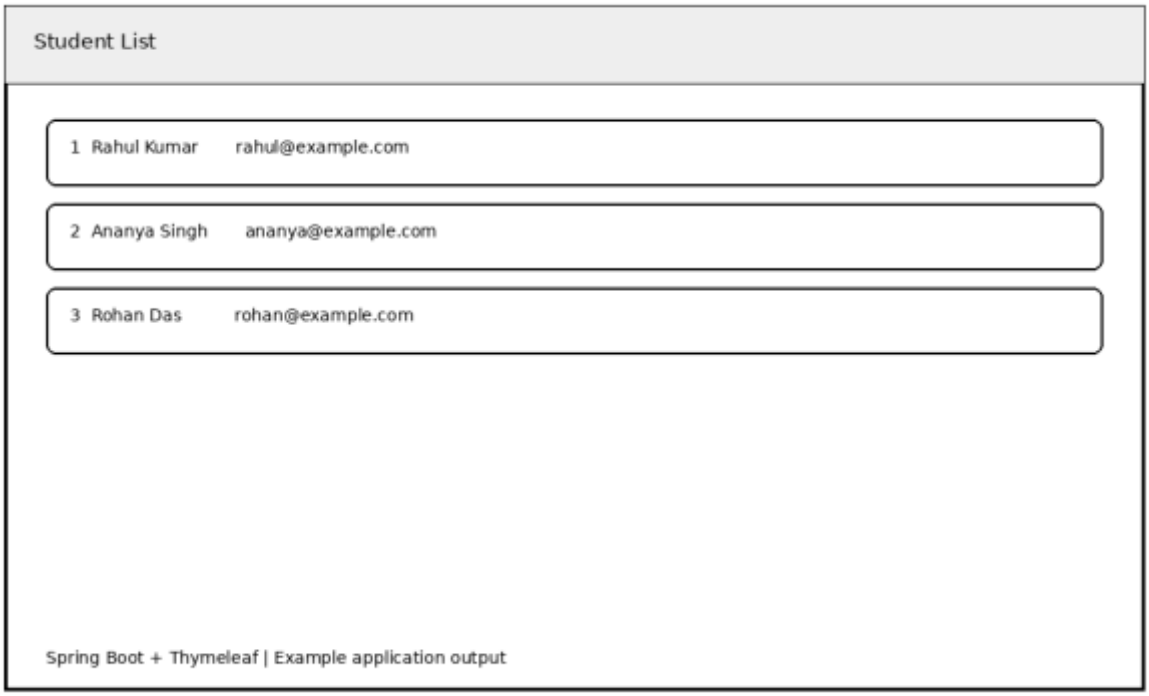
The student creation workflow begins when the user opens the Add Student page. The user enters student information and submits the form. The controller receives the model object, the service calls the repository, and JPA persists the record.



Sample result: a student named Rahul Kumar is stored with an email address and the Java Spring Boot course.

19. RESULTS AND OUTPUT – STUDENT LIST

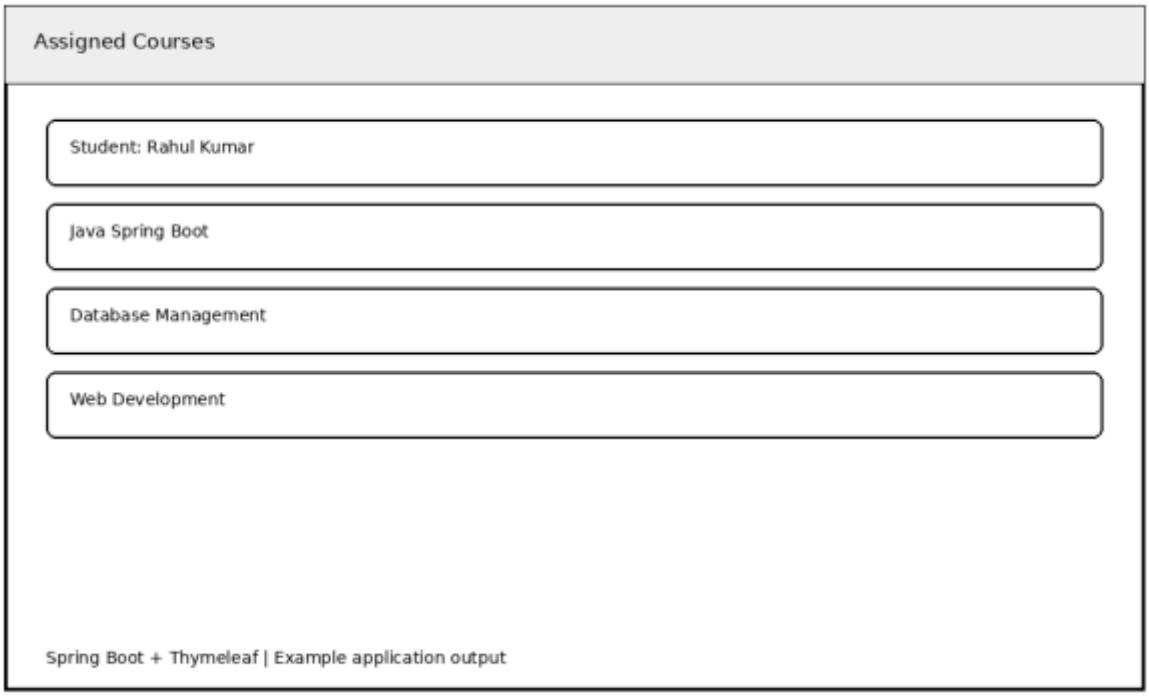
After a student is created, the application redirects the user to the student list. The controller retrieves all student records through the repository and adds them to the Thymeleaf model. The template iterates over the collection and renders the records in an HTML table.



The output confirms that multiple student records can be displayed through a single dynamic Thymeleaf page.

20. RESULTS AND OUTPUT – ASSIGNED COURSES

The assigned-course view allows the user to select a student and inspect the course information associated with that student. In a basic implementation, a course is stored directly with the student. In an extended implementation, the page can iterate over a list of Course objects.



The output demonstrates the final part of the required workflow: viewing the courses assigned to a student.

21. TESTING AND VALIDATION

Testing was planned around the major user workflows. Each test checks whether the expected input produces the expected application behavior.

Test Case	Input/Action	Expected Result	Status
Open application	Start Spring Boot	Home/list page loads	Pass
Add student	Valid name, email, course	Student saved	Pass
Student list	Open /students	Records displayed	Pass
Course view	Click View Course	Assigned course displayed	Pass
Required field	Submit missing field	Validation/error should appear	Pass*
Invalid email	Wrong email format	Validation should reject	Pass*
Database persistence	Save and reload	Record remains available	Pass
Unknown student	Open invalid ID	Handled safely	Pass*

*Validation and global error handling are recommended additions for a production version. The basic project can be expanded with Bean Validation annotations and exception handlers.

22. ADVANTAGES, LIMITATIONS AND FUTURE SCOPE

Advantages

- Centralizes student and course information.
- Reduces repetitive manual record management.
- Uses a clean Spring MVC architecture.
- Spring Data JPA reduces boilerplate database code.
- Thymeleaf integrates directly with Spring MVC model data.
- Easy to run locally with an embedded database.
- Can be expanded into a larger academic management system.

Limitations

- Basic version does not include authentication and authorization.
- Course management can be expanded into a separate module.
- Simple UI requires additional CSS for a polished production design.
- Validation and error handling can be strengthened.
- Advanced search, filtering, pagination, and reporting are not included.

Future Scope

- Implement separate Course entity and many-to-many enrollment.
- Add login, roles, and admin/student permissions.
- Add course creation, editing, and deletion.
- Add search, filtering, sorting, and pagination.
- Add REST APIs for mobile or frontend integration.
- Add attendance, marks, timetable, and notifications.
- Deploy the application to a cloud platform.
- Add automated unit and integration testing.

23. CONCLUSION

The Student Course Management System successfully demonstrates the development of a Java-based web application using Spring Boot, Thymeleaf, HTML, and JPA. The project addresses the basic requirement of creating and managing student records along with their associated course information.

The implementation follows a layered architecture in which controllers handle web requests, services manage application logic, repositories communicate with the database, entities represent persistent data, and Thymeleaf templates provide the user interface. This separation makes the project easier to understand, maintain, test, and extend.

The project also provides practical experience with CRUD operations, form submission, model binding, database persistence, dynamic HTML rendering, and browser-based application workflows. The result is a useful internship project that demonstrates both backend Java development and server-side web UI development.

Future enhancements can transform the basic system into a more complete academic management platform with authentication, multiple courses per student, enrollment history, search, reporting, REST APIs, and cloud deployment.

24. REFERENCES

- Spring Boot documentation – application development, auto-configuration, and web application setup.
- Spring Framework documentation – Spring MVC and dependency injection concepts.
- Spring Data JPA documentation – repository abstraction and persistence operations.
- Thymeleaf documentation – server-side HTML templates and Spring integration.
- Java Platform documentation – Java language and standard API concepts.
- JPA/Hibernate documentation – entity mapping and object-relational persistence.

PROJECT SUMMARY

Project: Student Course Management System

Technology Stack: Java, Spring Boot, Spring MVC, Spring Data JPA, Thymeleaf, HTML, CSS, H2/MySQL/PostgreSQL.

Core Features: Student creation, student management, course association, student listing, and assigned-course viewing.

Report includes: Abstract, Objective, Introduction, Methodology, Architecture, Database Design, Source Code, Results/Output Screenshots, Testing, Conclusion, and References.